Information Retrieval and Extraction : Lecture II

Roshni Chakraborty

Dept. Of Information Technology

ABV-IIITM Gwalior

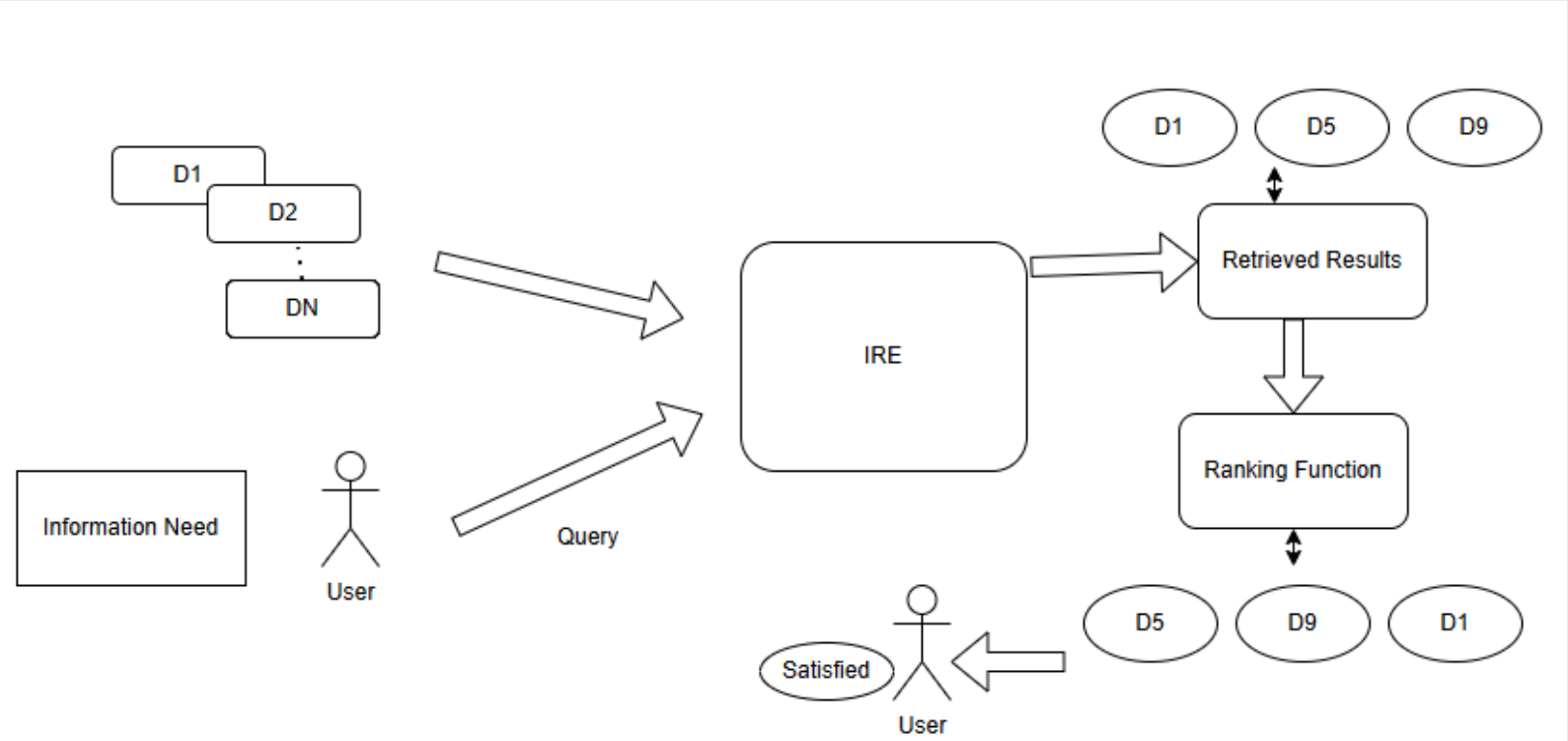
February 4, 2026

Outline I

1. Overview
2. Phrase Queries
   - Positional Queries
3. Processing Queries
   - Ranked Retrieval Models
   - Handling Typographical errors
   - Spelling correction

Overview

Recap



Pre-processing Details

Phrase Queries

- Query *Stanford University* should not match with
  - The inventor Stanford Ovshinsky never went to university
- How to support phrase queries and their combination?
  - Generate **bigrams**
- Consider every pair of consecutive terms in a document as a phrase
- *Friends, Romans, Countrymen* would generate biwords as
  - *Friends, Romans*
  - *Romans, Countrymen*
- *the abolition of slavery*, generate those bigrams
- There is always a chance of false positive matches
  - Increases the vocabulary size
  - leads to unnecessary words
- Solution : Generation of *keyphrases*

Pre-processing Details

Phrase Queries

- Consider, nouns and noun phrases
- Tokenize text and perform POS tagging
- Identify nouns (N)
  - proper nouns included
  - Consider joining words as articles, prepositions by (X)
  - accept any phrase as extended bigram, NX*N
- Target Identification/Index Creation
  - *cashless economy*, *Prime Minister of India*, *black money*, *Curb Inflation*, *Fight Corruption*, *long ATM queues*
- A false positive match on indexed phrases of length 3 is lower than
  - 2 and so on...
  - Increasing to trigrams, quadragrams can reduce false positives

Pre-processing Details

Positional Queries

- Include k words on either side
- Best case scenario
  - the most expensive phrase queries
  - individual words are common
  - but the desired phrase is comparatively rare
  - ex: *Sachin Tendulkar* is good but *Singh Is Kinng* , *The Who*
- If users commonly query on particular phrases, such as *Michael Jackson*,
  - quite inefficient to keep merging positional postings lists

Processing Queries

Intersection of Posting Lists

```
INTERSECT(p1, p2)
answer ← ⟨ ⟩
while p1 ≠ NIL and p2 ≠ NIL
do if docID(p1) = docID(p2)
      then ADD(answer, docID(p1))
           p1 ← next(p1)
           p2 ← next(p2)
      else if docID(p1) < docID(p2)
              then p1 ← next(p1)
              else p2 ← next(p2)
return answer
```

Processing Queries

Intersection of Posting Lists

- Efficiently intersect postings lists
- Merging the lists with a logical **AND** operation
- Query Optimization
  - process of selecting how to organize the work
  - reduce amount of work done in answering a query
  - examples : Brutus AND Caesar AND Calpurnia
    - start by intersecting the two smallest postings lists
    - ordering decision based on in-memory data

Processing Queries

Ranked Retrieval Models

- Users largely use *free text queries*
  - non-requirement of precise language
- Proximity Operator
  - way of specifying that two terms in a query appears **close**
  - Closeness is measured by by limiting the allowed number of intervening words
  - could be even same sentence/paragraph depends on window size

Processing Queries

Handling Typographical errors

- Ensure *robustness* to the query
  - Ensue handling of *alternative* spellings
  - Should consider handling of different variants of the same word
  - *Automat**
    - can match with *Automation*, *Automate*, ...,

Wildcard Queries

- User is uncertain of the spelling of a *query term*
  - Sydney vs. Sidney
  - Caesar vs. Ceasar
- User is aware of multiple variants of spelling a term
  - (consciously) seeks documents containing any of the variants
  - color vs colour
- Solution is *wildcard queries*

Processing Queries

Spelling corrections

- User wish to retrieve documents containing the term carrot
  - when the user types the query *carot*
- Solutions
  - based on edit distance
  - based on k-gram overlap
- For evert alternative correct spellings for a mis-spelled query
  - choose the "nearest" one

Things to Remember

- carot always retrieve documents containing carot as well as any "spell-corrected"
  - carrot and tarot
- when query **carot** does not exist
- when query **carot** exists but returns fewer documents than preset
  - then, it finds the spell corrected version and asks user

References

- Introduction to Information Retrieval by Christopher Manning (ch-2 and ch-3)
  - Ch2 : The term vocabulary and postings lists
  - Ch 3 : Dictionaries and tolerant retrieval

Thank you!